

Are You Getting What You Pay For?

Auditing Model Substitution in LLM APIs

Why Software Tests Fail and Hardware Attestation Wins

Based on: arxiv.org/abs/2504.04715

For ML researchers · AI security practitioners · Cloud infrastructure engineers

The Model Substitution Problem: Economic Incentives Drive Covert Swaps

The Core Problem

Commercial LLM API providers charge for access to specific models, but the black-box nature of APIs means users have no guarantee the advertised model is actually served.

Providers face strong economic incentives to covertly substitute cheaper alternatives to reduce GPU costs while maintaining pricing.

Formal Framework: Hypothesis Testing

H_0 : API serves $P_{\text{spec}}(y|x)$ — the specified model

H_1 : API serves $P_{\text{actual}}(y|x)$ — a substitute model

Type 1: Smaller Model

e.g., 8B served instead of 70B

Type 2: Quantized Variant

FP8/INT8 instead of FP16

Type 3: Fine-Tuned Version

Updated or modified weights

Type 4: Different Family

Entirely different architecture

FP8 Quantization Preserves Most Accuracy — Making Substitution Hard to Detect

Performance drops are small and often within error bars, motivating why detection is difficult.

Model	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B-Instruct	62.69 ± 0.18	61.14 ± 3.47	20.65 ± 3.43	22.62 ± 0.22
<i>Llama-3-8B-Instruct-FP8</i>	<i>62.43 ± 0.26</i>	<i>60.90 ± 4.10</i>	<i>14.91 ± 2.49</i>	<i>20.14 ± 0.24</i>
Llama-3-70B-Instruct	78.05 ± 0.08	88.06 ± 1.44	35.69 ± 1.33	29.60 ± 0.51
<i>Llama-3-70B-Instruct-FP8</i>	<i>77.88 ± 0.13</i>	<i>87.35 ± 1.24</i>	<i>35.75 ± 1.16</i>	<i>33.12 ± 0.30</i>
Gemma-2-9b-it	71.86 ± 0.08	81.80 ± 1.35	33.41 ± 0.28	28.64 ± 2.97
<i>Gemma-2-9b-it-FP8</i>	<i>71.92 ± 0.11</i>	<i>79.41 ± 1.14</i>	<i>32.53 ± 0.34</i>	<i>27.81 ± 3.22</i>
Qwen2-72B-Instruct	82.18 ± 0.08	86.72 ± 1.00	37.39 ± 1.41	29.93 ± 2.71
<i>Qwen2-72B-Instruct-FP8</i>	<i>81.98 ± 0.08</i>	<i>86.82 ± 0.97</i>	<i>37.67 ± 1.39</i>	<i>31.08 ± 1.96</i>
Mistral-7B-Instruct-v0.3	59.15 ± 0.10	35.90 ± 4.54	8.94 ± 1.24	21.60 ± 0.17
<i>Mistral-7B-v0.3-FP8</i>	<i>58.77 ± 0.13</i>	<i>32.20 ± 4.02</i>	<i>7.68 ± 1.23</i>	<i>22.72 ± 0.19</i>

 Key Insight:

FP8 scores frequently fall within the error bars of the original precision model, especially on MMLU and GSM8K.

Higher Temperature Amplifies Performance Gaps but Increases Noise

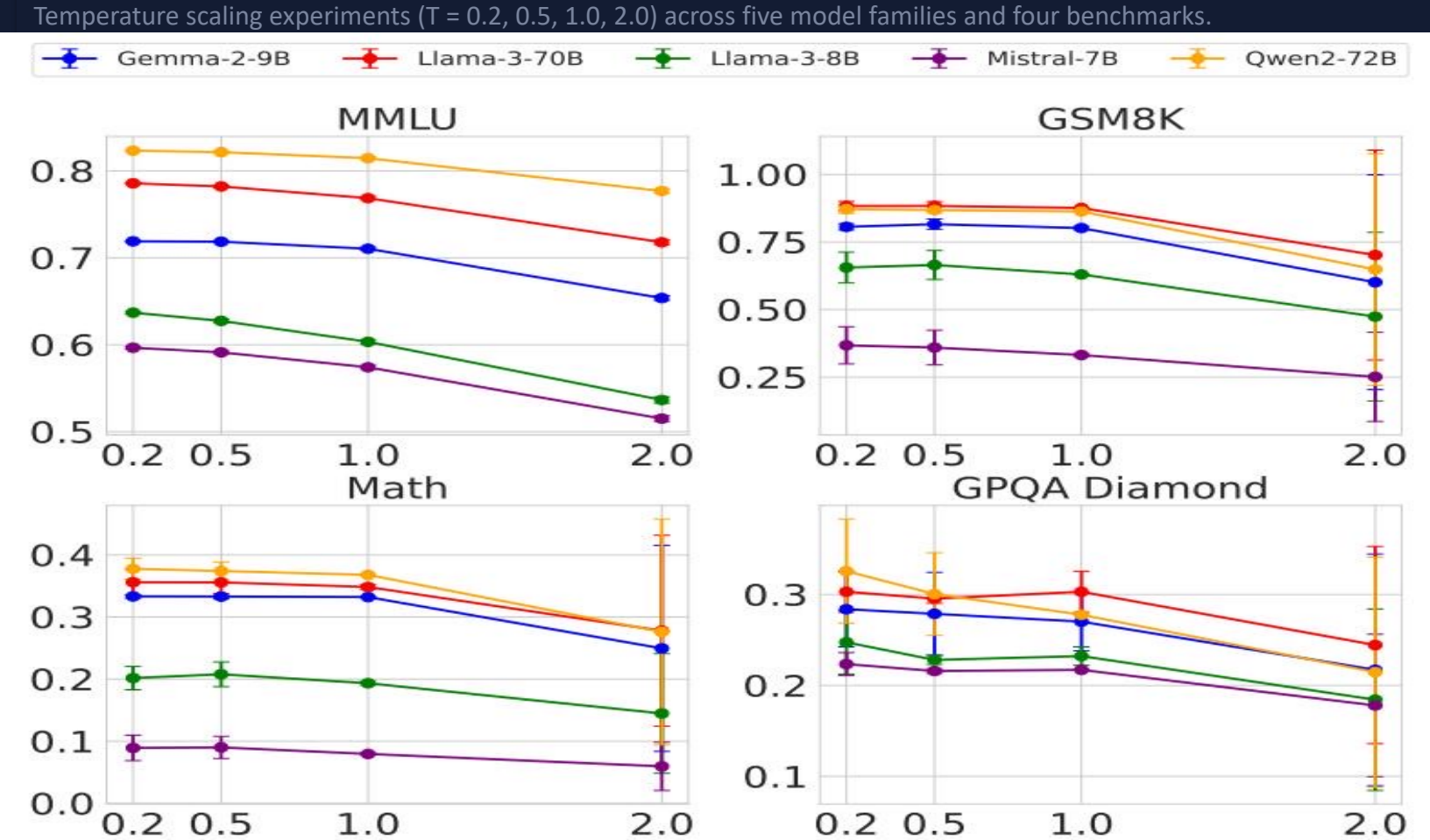


Figure 1: Benchmark accuracy vs. temperature for five models across four tasks (source_brief.md)

Low Temperature (0.2)

Models perform near peak. Subtle differences between original and FP8 are most visible here.

High Temperature (2.0)

All models degrade significantly. The gap between original and substitute narrows or becomes noisy.

Detection Implication

Higher temperatures further obscure the substitution signal, making statistical tests even less reliable.

Log Probability Fingerprinting Fails Due to Inference Nondeterminism

The same model produces different log probabilities across software versions and GPU hardware.

Model	BERT Acc	T5 Acc	GPT2 Acc	LLM2Vec Acc
Llama3-70B-Instruct-FP8	50.55	50.10	50.45	49.90
Llama3-70B-Instruct-INT8	51.60	49.90	51.30	50.25
Gemma2-9b-it-FP8	49.95	50.30	51.20	49.80
Gemma2-9b-it-INT8	49.00	49.70	51.65	49.55
Mistral-7b-v3-Instruct-FP8	50.55	49.75	48.70	48.75
Mistral-7b-v3-Instruct-INT8	49.70	50.75	50.50	51.15
Qwen2-72B-Instruct-FP8	50.05	50.25	48.75	49.80
Qwen2-72B-Instruct-INT8	50.75	50.55	49.45	50.20

Software Variation

Transformers 4.40 vs 4.50 and vLLM 0.6.2 vs 0.8.2 produce different log-probs for identical inputs.

Hardware Variation

A100 vs H100 GPUs yield measurably different log-probability distributions for the same Llama-3-8B model.

Result

Auditors cannot distinguish between a legitimate model on different infrastructure and an actual substitution.

Figure 5: Log-probability variation across inference frameworks and hardware for Llama-3-8B

Randomized Routing at Low Rates Makes Statistical Detection Query-Prohibitive

Specified Model

$P_{\text{spec}}(y|x)$
Probability: $(1 - p)$

Randomized Router

$P_{\text{mixed}} = (1-p) \cdot P_{\text{spec}} + p \cdot P_{\text{alt}}$
As $p \rightarrow 0$, $P_{\text{mixed}} \rightarrow P_{\text{spec}}$

Substitute Model

$P_{\text{alt}}(y|x)$
Probability: p

The Attack

Provider routes a fraction p of queries to the cheaper model. The remaining $(1-p)$ queries go to the genuine model.

Benchmark Evasion

Providers can detect known benchmark prompts and route them to the genuine model, serving substitutes only for regular traffic.

Consequence

As p decreases, the mixed distribution converges to the specified model's distribution, requiring exponentially more queries to detect — making software-only detection fundamentally query-prohibitive.

Software-Only Auditing Methods Are Fundamentally Unreliable

All three detection paradigms fail under realistic conditions.

Method Type	Approach	Core Weakness	Practical Limitation
Text-Based Statistical Tests	MMD and distributional tests on model outputs	Requires impractically large query budgets	Fails against subtle or randomized substitutions
Log-Probability Fingerprinting	Compare token-level log-probs vs. reference	Defeated by production-level inference nondeterminism	Software versions and GPU hardware shift log-probs
Adversarial Countermeasures	Detect and evade audit queries	Benchmark evasion routes test prompts to real model	Randomized routing makes detection query-prohibitive

Verdict

- Statistical tests on text outputs (e.g., MMD) need impractically large sample sizes and fail against subtle substitutions.
- Log-probability fingerprinting is unreliable due to inherent nondeterminism across inference stacks.
- Adversarial countermeasures (randomized routing, benchmark evasion) further undermine all software-only approaches.

Trusted Execution Environments Provide Provable Cryptographic Model Integrity

What are TEEs?

Trusted Execution Environments are hardware-secured enclaves that provide cryptographic attestation of the code and data being executed. With TEEs, the provider can prove that specific model weights are loaded and that the correct inference code is running.

Provable

Cryptographic attestation guarantees model identity. No trust-me assumptions.

Efficient

Only modest performance overhead vs. unprotected inference.

Robust

Resistant to all software-based evasion techniques identified in this work.

✗ Software Methods

Unreliable, query-prohibitive, easily evaded

✓ TEEs (e.g., NVIDIA H100 Confidential Computing)

Provable, efficient, practical and deployable today

Note: Zero-knowledge proofs (ZKPs) are computationally prohibitive for large models; TEEs are practical today.

Key Takeaways: Close the Verification Gap with Hardware Attestation

1

Model substitution is economically motivated and practically undetectable via software — providers save GPU costs while users pay full price.

2

FP8 quantization barely affects benchmark scores — performance drops are often within error bars, making substitution nearly invisible.

3

Inference nondeterminism across software versions and GPU hardware defeats log-probability fingerprinting.

4

Randomized substitution and benchmark evasion make all software-only detection approaches fundamentally query-prohibitive.

5

Trusted Execution Environments are the only currently viable path to verifiable LLM API integrity — provable, efficient, deployable today.

Call to Action: Demand hardware-attested inference from your LLM API providers. The tools exist — adoption is the remaining barrier.